

EnunciadoJunio2024.pdf



Sarcasmmodeon



Estructuras de Datos Avanzadas



3º Grado en Ingeniería Informática



**Escuela Técnica Superior de Ingeniería Informática.
Campus de Vicálvaro
Universidad Rey Juan Carlos**



[Accede al documento original](#)

 **Gemini**

**Dale al play y
estudia a tu ritmo**

Convierte estos apuntes en un track pegadizo
que me ayude a memorizar las palabras clave

Apuntes Tema 3

 PDF



Razonamiento ▾



Pruébalo

Máster en Inteligencia Artificial y Ciencia de Datos



Impulsa tu perfil
en IA y datos

Más info

Estructuras de Datos Avanzadas Examen convocatoria de septiembre 18 de junio de 2024

- La duración del examen es de 120 minutos.
- Desde el Aula Virtual hay que descargar un archivo ZIP que contiene el esqueleto de los ejercicios del examen así como los test unitarios correspondientes. Se recomienda crear un proyecto de Apache Netbeans vacío; copiar dentro de la carpeta del proyecto la carpeta src y la carpeta test proporcionadas; añadir las librerías JUnit4 y Hamcrest; y finalmente cerrar y volver a abrir Apache Netbeans.
- El estudiante debe generar un fichero ZIP con el directorio en el que esté el código elaborado por él/ella y deberá entregar dicho fichero ZIP en la actividad examen práctico del Aula Virtual asegurándose de que entrega el código desarrollado durante la prueba.

Nombre y Apellidos: _____

Hora de entrega: _____

Ejercicio 1 HyperGraph [4 puntos]

Un hipergrafo es un grafo cuyas aristas relacionan dos o más nodos entre sí. Además, estas aristas no pueden contener nodos repetidos (no se consideran bucles). Deseamos realizar la implementación de un hipergrafo. En la carpeta `material.graphs` encontrará la interfaz `HyperGraph` y la clase `ELHyperGraph` que cumple dicha interfaz.

En la clase `ELHyperGraph` debe definir algunas propiedades de la clase privada `ELEdge` y también las operaciones **[0,5 puntos]**:

- a) constructor de clase `ELEdge`
- b) operación `getNodos`
- c) operaciones `hashCode` e `equals`. Deben emplear todas las propiedades de la clase `ELEdge`.

De la clase `ELHyperGraph` implemente las operaciones:

- d) **[0,5 puntos]** `others`, que dada una arista y un vértice que debe formar parte de la arista, devuelve una colección inmodificable de los vértices que componen la arista, sin incluir el que se pasa como parámetro.
- e) **[0,5 puntos]** `endVertices`, que dada una arista devuelve una colección inmodificable de los vértices que forman parte de la arista.
- f) **[1 punto]** `getAdjacent`, que dados dos vértices, devuelve una colección inmodificable con todas las aristas en las que están presentes ambos vértices.
- g) **[0,75 puntos]** `insertEdge`, que inserta una nueva arista en el hipergrafo.
- h) **[0,75 puntos]** `removeEdge`, que elimina del grafo la arista que se le pasa como parámetro.

NOTA: La clase `ELHyperGraph` y la interfaz `HyperGraph` se encuentran en la carpeta `material.graphs`. El estudiante puede agregar todos los métodos privados que considere necesarios pero no está permitido modificar las cabeceras de los métodos suministrados o agregar métodos adicionales públicos.

WUOLAH



Ejercicio 2. Syntetic_Intel [6 puntos]

La empresa Syntetic_Intel ha sufrido un crecimiento demasiado rápido en los últimos meses. Tanto que sus antiguos sistemas de recursos humanos se han visto desbordados hasta el punto de no saber quién es el jefe de quién. Cuando eran pocos empleados lo gestionaban con una lista de pares “empleado, jefe”, pero ahora ese listado es inmanejable, entre otras cosas porque un “jefe” es responsable de un número indeterminado de empleados. Por ello han decidido desarrollar un sistema más moderno que les permita, empleando las mismas fichas de los empleados, representar jerárquicamente el organigrama de la empresa, y de esta manera saber quién es el jefe de quién. Un jefe puede estar a cargo de múltiples empleados pero un empleado solo tiene un jefe directo.

Para abordar el cambio de sistema han decidido plantear dos fases. En una primera fase quieren utilizar el antiguo listado de pares “empleado,jefe” para generar una estructura intermedia que permita conocer, con coste $O(1)$, todos los empleados que tiene asignado cada jefe.

Es importante hacer notar que el ID asociado a cada empleado es único.

Una vez finalicen la primera fase, emplearán esa estructura intermedia para construir el organigrama de la empresa.

a) **[2 puntos]** Implemente la clase `NewBasicED`, eligiendo la estructura de datos adecuada para que la operación `team`, que recibe el identificador de un jefe y retorna todos los empleados que están directamente a su cargo tenga coste $O(1)$.

Implemente la clase `Syntetic_Intel`, se valorará positivamente que la codificación de los métodos sea legible y que la complejidad sea la mínima posible:

b) **[1 punto]** Después de leer completamente el enunciado elija las estructuras de datos necesarias para resolver el ejercicio. Implemente el constructor de la clase `Syntetic_Intel`.

c) **[2 punto]** Implemente el método `levelManagers` que recibe un valor entero y devuelve un `Iterable` con todos los managers que tengan ese nivel dentro del organigrama de la empresa.

d) **[1 punto]** Implemente el método `allMyManagers` que recibe el identificador de un empleado y devuelve un `Iterable` con todos los jefes que tiene dicho empleado, el último jefe siempre será el CEO de la empresa. En caso de que el identificador del empleado se corresponda con el CEO el `Iterable` se devolverá vacío.

NOTA: Las clases `NewBasicED` y `Syntetic_Intel` se encuentran en la carpeta `examen`. En esta misma carpeta se incluye la clase `ID` que representa la información de cada empleado. El estudiante puede agregar todos los métodos privados que considere necesarios pero no está permitido modificar las cabeceras de los métodos suministrados o agregar métodos adicionales públicos.